The Journal of Open Source Software

GLPlot: GPU-accelerated interactive scientific visualization with a Matplotlib-like interface*

Juan Manuel Lombardi [1¶], **Felix Riccius** [1*], **Julian Holland** [1*], and **Gianmarco Ducci**[1*]

1 Fritz Haber Institute of the Max Planck Society, Berlin, Germany ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Interactive visualization is part of the scientific reasoning process: researchers use plots to inspect convergence, identify outliers, compare simulations, assess uncertainty, and decide which analyses or experiments to perform next. These tasks become slow when a figure contains very many points, vertices, curves, or other geometric primitives, because conventional plotting workflows pay for CPU-side geometry construction, per-object draw calls, repeated rasterization, and the transfer of expanded geometry to the graphics device. Waiting on every view change is more than an irritation: fewer lines of reasoning are pursued when each one is slow to check.

Here, we propose `GLPlot`, an open-source Python plotting package that exposes a familiar `pyplot`-style interface while using an OpenGL rendering engine in the backend. It is intentionally developed to support common two- and three-dimensional scientific visualizations, including polylines, scatter plots, histograms, matrix images, contour and pseudocolor plots, vector fields, surfaces, and meshes. A more dedicated `plot_lines` interface represents large families of straight lines by their slope and intercept coefficients and expands their visible geometry analytically on the GPU.

The rendering engine separates scene representation from GPU execution. It combines event-driven updates, shader-based geometry generation, stable viewport transformations, hardware clipping, density accumulation, adaptive level-of-detail control, and GPU-based picking. These operations are exposed to the user through a high-level Python interface, allowing large scientific datasets to be explored without direct interaction with low-level OpenGL components.

Statement of need

GPU-oriented visualization libraries can provide high rendering throughput, but they often require users to adopt a different programming model or work closer to graphics concepts than is desirable in routine scientific analysis (Campagnola et al., 2013; fastplotlib Developers, 2022). Conversely, pixel-based aggregation systems efficiently summarize very large datasets, but do not necessarily preserve the semantics or interactive selection of individual geometric objects (Datashader Developers, 2016; Stevens et al., 2015). `GLPlot` targets the space between these approaches. The intended users are computational scientists, engineers, and data analysts who require interactive rendering of large two- and three-dimensional datasets while retaining a concise interface close to the idiom of standard plotting libraries such as Matplotlib. The software is particularly suited to cases where the plotted entities remain meaningful geometric objects, such as line families in chemical-potential diagrams, feasibility diagrams, trajectory

*Software repository: https://github.com/AkarisDimitry/GLPlot

DRAFT

ensembles, vector fields, meshes, and selectable point clouds. The design is guided by five requirements:

1. avoid CPU-side geometry expansion when equivalent geometry can be generated analytically on the GPU;
2. maintain stable coordinate transforms during deep zoom operations at large coordinate offsets;
3. preserve density information when many primitives overlap;
4. avoid unnecessary CPU and GPU work when a scene is unchanged; and
5. support interactive inspection and selection without CPU searches that scale linearly with the number of rendered objects.

State of the field

Scientific visualization tools span a range of abstraction levels and rendering strategies; general-purpose plotting libraries, browser-based libraries (Bokeh Development Team, 2018; Plotly Technologies Inc., 2015), VTK-based applications (Ahrens et al., 2005; Ramachandran & Varoquaux, 2011; Schroeder et al., 2006), GPU frameworks, and aggregation systems answer different needs and complement rather than replace one another.

Matplotlib is the reference plotting library of scientific Python, with static, animated, and interactive figures and extensive layout, annotation, and backend systems (Hunter, 2007). Its artist-based architecture coordinates geometry construction, coordinate transforms, and redraws largely on the CPU, so performance deteriorates for very large numbers of primitives: dense line families, large point clouds, trajectory ensembles, and three-dimensional scenes updated interactively.

VisPy offers high-performance OpenGL visualization, from buffers and shaders up to scene graphs and reusable visuals (Campagnola et al., 2013). This flexibility suits custom applications, but specialized behavior often requires working with graphics concepts or writing custom visuals, a substantial burden for routine plotting; fastplotlib makes a similar trade with a modern GPU API (fastplotlib Developers, 2022).

Datashader rasterizes and aggregates data into the display's pixel grid (Datashader Developers, 2016), which preserves the distribution of severely overplotted data; the aggregate representation, however, does not keep individual lines, points, or surfaces as persistent objects that can be selected or styled.

PyQtGraph targets responsive desktop graphics for instrumentation and rapidly updated signals (Campagnola & PyQtGraph Contributors, 2012); its strength is fast two-dimensional application development rather than shader-side primitive generation, density accumulation, or GPU object selection in three dimensions.

`GLPlot` sits between these approaches. Contributing these capabilities to an existing package would have meant changing its core rendering model, so `GLPlot` keeps a procedural interface close to Matplotlib's `pyplot` and builds a GPU-resident architecture underneath it, for workloads in which many geometric objects must remain individually represented, interactive, and identifiable without a custom graphics pipeline.

Software design

`GLPlot` converts plotting data into OpenGL-ready buffers on the CPU and delegates the expensive per-frame work to the GPU: 2D data is staged as float32 vertex buffers and rendered through shader-based wide-line and density-accumulation paths; 3D data is built into geometry buffers with model-view-projection transforms and rendered with perspective scaling, depth testing, ambient-occlusion/rim shading, and transparency composition. CPU readback is

DRAFT

restricted to explicit export calls through `glReadPixels`; the interactive path never transfers pixels back to the host. Rendering uses OpenGL (Segal & Akeley, 2019) through PyOpenGL (Fletcher, 2005) and GLFW (Löwy & GLFW Contributors, 2002), and the interactive editing workstation is built with Dear ImGui (Cornut, 2014).

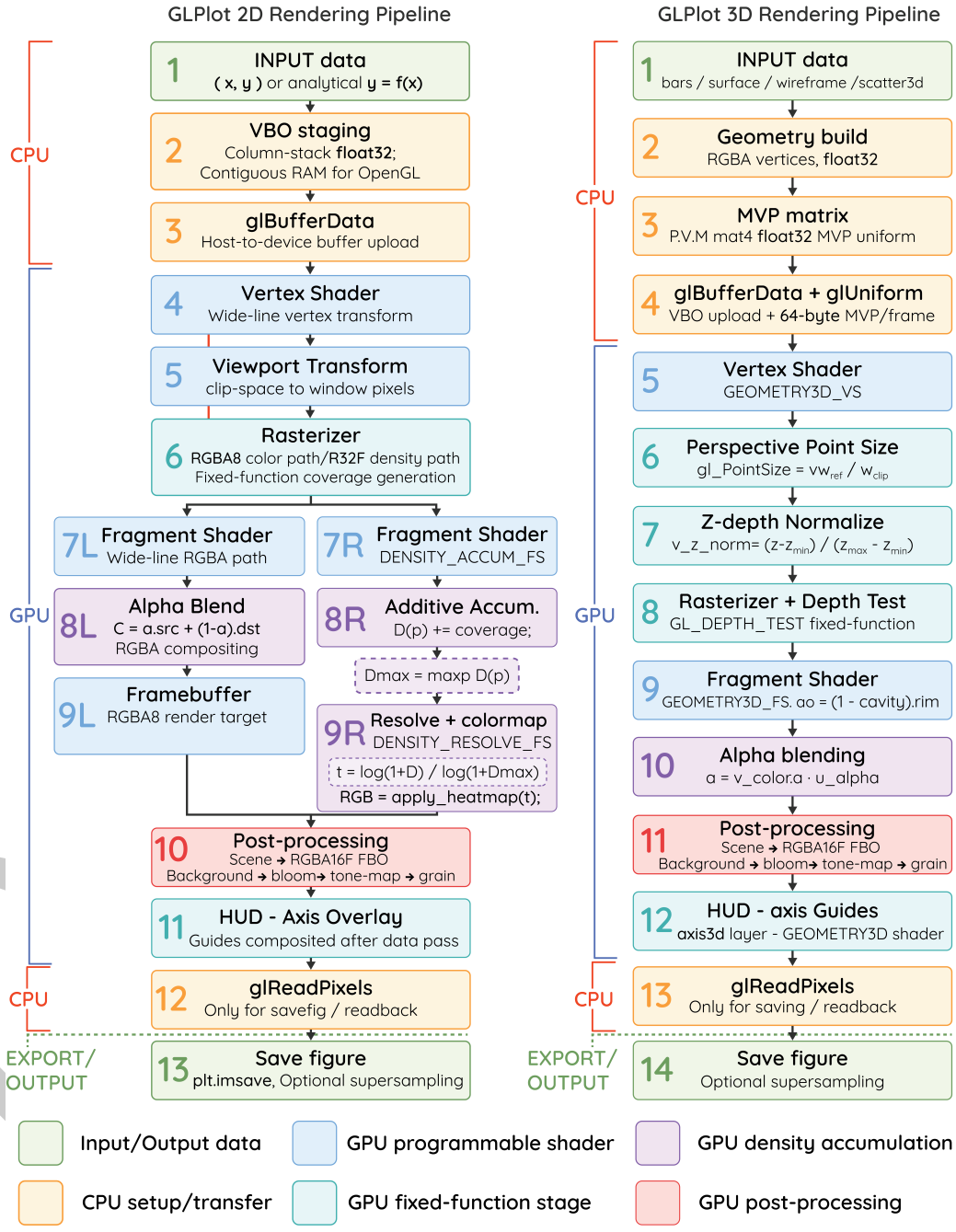


Figure 1: GLPlot's 2D and 3D rendering pipelines, from CPU-side data staging through GPU shading, blending, and post-processing to optional export.

User-facing interface

`GLPlot` is available on PyPI and installs with `pip install glplot`. The primary entry point is `glplot.pyplot`. Calls such as `plot`, `scatter`, `hist2d`, `pcolormesh`, `contour`, `quiver`,

`plot_surface`, `mesh3d`, and `bar3d` create layers in a scene, accept NumPy arrays (Harris et al., 2020), and follow common Matplotlib calling conventions, including format strings for line and marker styling; select numerical routines use SciPy (Virtanen et al., 2020). A more specialized `plot_lines` interface instead takes a line family as coefficient pairs:

```
import numpy as np
import glplot.pyplot as plt

slopes = np.random.default_rng(0).normal(size=1_000_000)
intercepts = np.random.default_rng(1).normal(size=1_000_000)

plt.figure()
plt.plot_lines(slopes, intercepts, x_range=(-5.0, 5.0))
plt.set_cmap("magma")
plt.density_gain(0.8)
plt.xlabel("x")
plt.ylabel("a x + b")
plt.show(density=True)
```

This data model reduces host-side geometry construction: rather than the CPU generating per-line endpoints, the active viewport determines each line's visible segment analytically in the vertex shader.

Rendering architecture and precision

Data arrays, styles, coordinate transforms, and renderer-specific metadata are stored as scene layers; a rendering manager maps each layer to a primitive renderer and the framebuffer passes it needs. The render loop is reactive, redrawing only when a layer, camera transform, or effect is marked dirty, so an unchanged figure costs nothing between events.

For a line family $y_i(x) = a_i x + b_i$, `GLPlot` uploads (a_i, b_i) as instanced attributes; the vertex shader analytically recomputes each line's visible endpoints against the current viewport bounds and discards out-of-domain instances before rasterization, rather than the CPU generating explicit per-line geometry. To stay numerically stable during deep zoom, the viewport center is computed in double precision on the CPU, and world coordinates are expressed relative to it before conversion to normalized device coordinates — keeping the values the GPU sees close to the display range regardless of the absolute coordinate offset.

Density, interaction, and level of detail

Dense, overlapping geometry is accumulated additively into a floating-point framebuffer, log-normalized to the display range, and optionally tone-mapped in the style of Reinhard et al. (2002) to preserve contrast. During rapid pan or zoom, a cached offscreen image is reprojected over the expanded viewport and an exact redraw is scheduled once interaction stops, so pointer motion never forces full geometry work. Level of detail is decided from estimated fill-rate cost rather than primitive count alone, using viewport size, line width, and projected polyline length; because the decision is derived from the instance identifier, a static view does not flicker between frames. Object selection uses a deferred integer picking pass: selectable primitives write an encoded identifier to an integer framebuffer, and a click resolves to a layer and element index from a single-pixel readback, independent of scene size.

Research impact statement

`GLPlot` is designed for research workflows in which visualization is a bottleneck between numerical computation and interpretation. It has been used in this capacity in `EZGA`, a published evolutionary structure-exploration framework for computational materials science, to

DRAFT

visualize energy landscapes, convex-hull reconstructions, and surface phase diagrams produced across its benchmark systems (Lombardi et al., 2026). More broadly, representative applications include the inspection of large trajectory projections, chemical-potential line families, phase-stability or feasibility constructions, energy distributions, structural descriptors, spatial fields, large vector fields, and uncertainty ensembles. Related visualization patterns occur in atomistic simulation, computational chemistry and physics, bioinformatics, engineering simulation, and exploratory data analysis.

The principal scholarly contribution is to make established GPU rendering techniques, combined as described above, usable by researchers who should not need to manage shaders and framebuffer objects themselves.

Limitations

`GLPlot` focuses on high-performance interactive visualization rather than exhaustive Matplotlib compatibility. Complex publication layouts, specialized axis artists, advanced tick formatting, and some annotation or backend features may remain better served by Matplotlib. GPU rendering also introduces hardware and driver dependencies, so performance reports should record the GPU, driver and OpenGL versions, operating system, and viewport size.

The package requires an OpenGL 3.3 core-profile context, the minimum its shaders request; figures can also be exported headlessly. Continuous integration runs the test suite on Linux, macOS, and Windows with Python 3.11 to 3.13, using Mesa's software OpenGL under a virtual display on Linux.

AI usage disclosure

Generative AI was used in developing `GLPlot`. Claude Opus 5.5 (Anthropic), through Claude Code, and OpenAI Codex were used as coding assistants for parts of the implementation, the test suite, packaging and continuous-integration configuration, documentation and examples, and for editing this manuscript. The authors made the core design decisions, defined the problems to be solved, and reviewed, tested, and validated all AI-assisted output: changes were accepted only after passing the automated test suite and manual inspection of the rendered figures. The authors take full responsibility for the software and this paper.

Author contributions

J.M.L. conceived and developed the software, wrote the documentation and the test suite, and drafted the manuscript. F.R., J.H., and G.D. contributed equally to the writing of the manuscript, to the design of features, and to testing the software in real research applications.

Acknowledgements

This work was supported by the Fritz Haber Institute of the Max Planck Society, which had no role in the design of the software or in the writing of this paper.

Conflicts of interest

The authors declare no conflicts of interest.

DRAFT

References

Ahrens, J., Geveci, B., & Law, C. (2005). ParaView: An end-user tool for large data visualization. In *Visualization handbook*. Elsevier. ISBN: 978-0-12-387582-2

Bokeh Development Team. (2018). *Bokeh: Python library for interactive visualization*. https://bokeh.org.

Campagnola, L., Klein, A., Larson, E., Rossant, C., & Rougier, N. P. (2013). *VisPy: Interactive scientific visualization in Python*. https://vispy.org.

Campagnola, L., & PyQtGraph Contributors. (2012). *PyQtGraph: Scientific graphics and GUI library for Python*. https://www.pyqtgraph.org.

Cornut, O. (2014). *Dear ImGui: Bloat-free graphical user interface for C++*. https://github.com/ocornut/imgui.

Datashader Developers. (2016). *Datashader: Accurate, fast visualization of large datasets*. https://datashader.org.

fastplotlib Developers. (2022). *fastplotlib: Next-generation GPU-accelerated scientific visualization*. https://github.com/fastplotlib/fastplotlib.

Fletcher, M. C. (2005). *PyOpenGL: The python OpenGL binding*. https://pyopengl.sourceforge.net.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Lombardi, J. M., Riccius, F., Pare, C. W. P., Heenen, H. H., Reuter, K., & Scheurer, C. (2026). EZGA: An evolutionary structure exploration framework. *Computer Physics Communications*, 110354. https://doi.org/10.1016/j.cpc.2026.110354

Löwy, C., & GLFW Contributors. (2002). *GLFW: A multi-platform library for OpenGL, OpenGL ES, and Vulkan development*. https://www.glfw.org.

Plotly Technologies Inc. (2015). *Collaborative data science*. https://plot.ly.

Ramachandran, P., & Varoquaux, G. (2011). Mayavi: 3D visualization of scientific data. *Computing in Science & Engineering*, *13*(2), 40–51. https://doi.org/10.1109/MCSE.2011.35

Reinhard, E., Stark, M., Shirley, P., & Ferwerda, J. (2002). Photographic tone reproduction for digital images. *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '02)*, 267–276. https://doi.org/10.1145/566570.566575

Schroeder, W., Martin, K., & Lorensen, B. (2006). *The visualization toolkit: An object-oriented approach to 3D graphics* (4th ed.). Kitware, Inc. ISBN: 978-1-930934-19-1

Segal, M., & Akeley, K. (2019). *The OpenGL graphics system: A specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf.

Stevens, J.-L. R., Rudiger, P., & Bednar, J. A. (2015). HoloViews: Building complex visualizations easily for reproducible science. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 59–66. https://doi.org/10.25080/Majora-7b98e3ed-00a

DRAFT

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT